

Exercise- 1:

SingletonPattern.cs

```
using System;

class Logger
{
    private static Logger instance;
    // Private constructor
    private Logger(){
    }
    // Public method to get the single instance
    public static Logger GetInstance(){
        if (instance == null)
        {
            instance = new Logger();
        }
        return instance;
    }
    public void DisplayMessage() {
        Console.WriteLine("Logger Instance Created");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Logger logger1 = Logger.GetInstance();
        Logger logger2 = Logger.GetInstance();
        logger1.DisplayMessage();
        if (logger1 == logger2)
        {

```

```

        Console.WriteLine("Both objects are the same instance.");
    }
    else
    {
        Console.WriteLine("Different instances created.");
    }
    Console.ReadLine();
}
}

```

Output:

```

PS C:\Users\suman\OneDrive\Desktop\CTS\SingletonPattern> dotnet run
C:\Users\suman\OneDrive\Desktop\CTS\SingletonPattern\Program.cs(5,27): warning CS8618: Non-nullable
hen exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable
Logger Instance Created
Both objects are the same instance.

```

Exercise – 2:

FactoryMethodPattern.cs

```
using System;
```

```
// Product Interface
```

```
interface IShape
```

```
{
    void Draw();
}
```

```
// Concrete Product 1
```

```
class Circle : IShape
```

```
{
    public void Draw()
    {
        Console.WriteLine("Drawing Circle");
    }
}
```

```
    }  
}
```

```
// Concrete Product 2
```

```
class Rectangle : IShape
```

```
{  
    public void Draw()  
    {  
        Console.WriteLine("Drawing Rectangle");  
    }  
}
```

```
// Factory Class
```

```
class ShapeFactory
```

```
{  
    public IShape GetShape(string shapeType)  
    {  
        if (shapeType.Equals("Circle", StringComparison.OrdinalIgnoreCase))  
        {  
            return new Circle();  
        }  
        else if (shapeType.Equals("Rectangle", StringComparison.OrdinalIgnoreCase))  
        {  
            return new Rectangle();  
        }  
        else  
        {  
            throw new ArgumentException("Invalid Shape Type");  
        }  
    }  
}
```

```
}

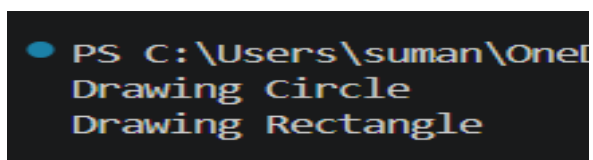
// Main Class
class Program
{
    static void Main(string[] args)
    {
        ShapeFactory factory = new ShapeFactory();

        IShape shape1 = factory.GetShape("Circle");
        shape1.Draw();

        IShape shape2 = factory.GetShape("Rectangle");
        shape2.Draw();

        Console.ReadLine();
    }
}
```

Output:



```
PS C:\Users\suman\OneD
Drawing Circle
Drawing Rectangle
```